1) Git = open src software to track code timeline

4) GitHUb = web platforms that host git repo

---

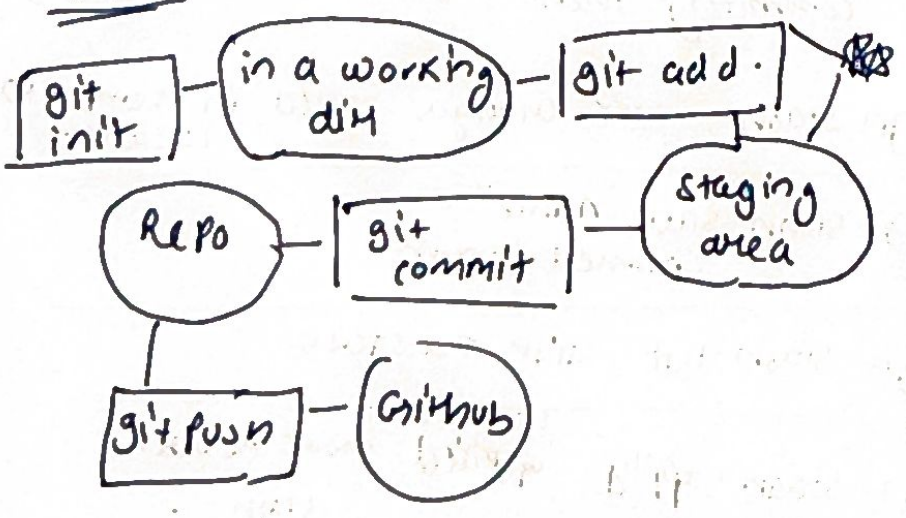
installation

1) install git (Software)
2) git --version
3) git config -- global user.name
4) git config -- global user.email.

---

1) mkdir s ← creates a directory
2) cd s ← changes directory
3) git init ← creates
4) type nul > f.txt (make a new file)
5) notepad f.txt (edit it)
6) dir (check all files)
7) git status (Show untracked files)
8) git add . or git add (file name)
9) git commit -m "msg"
10) git log (shows commit history)

Flow



Internal working of git:

(1) Git snapshots

History of all the code saved in a key val pair

Key → unique hash code (string)

Value → object

3 parts

(1) commit object: Each commit stored in git folder in form of commit object

(2) Tree obj → files/folder

(3) Blob object ⇒ actual code

commit obj
-) Tree obj
-) Parent commit obj
-) Author
-) Committor
-) commit msg

Tree obj
-) File name
-) file type
-) file mode
-) pointer tree object

↓

blob

actual code

Branches

1) git branch x42 (create branch x42)
2) git branch shows curr branch
3) git checkout x42 switch to x42 branch
4) git merge x42 (merges x42 to curr branch

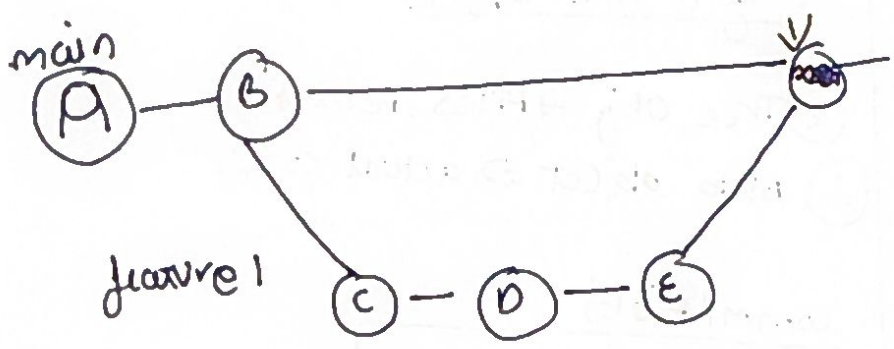
Merging

① Fast forward merge (Not diverged)
② 3 way merge (diverged)

1) Fast forward merge

A branch (main) has no commits & other does → 0 conflicts
Easy merge

~~2) 3 way merge~~



the main branch had no commits after creation of feature branch
easy merge

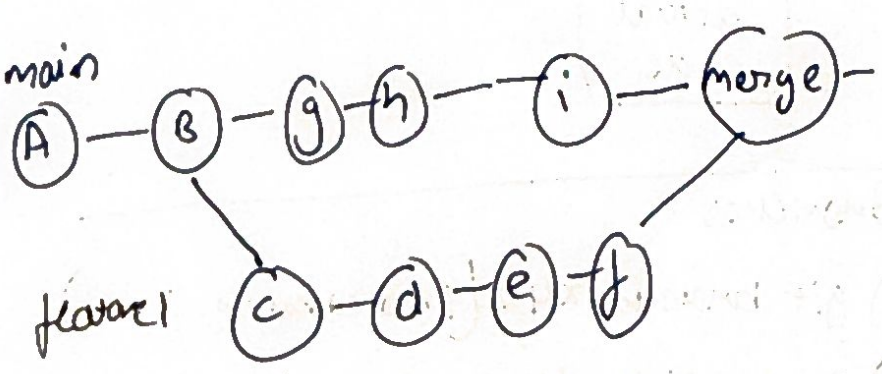
```
git checkout main
git merge feature1
```

---

② 3 way Merge

#SAME code

→ But there are conflicts & u have to Resolve them



This will give conflicts as main have many commits not present in feature branch

---

Git diff : compares changes made in one commit & other commit.

Git consider same file into 2 files
① original file → a/
② updated file → b/

Then compares it

--- → start of a/
+++ → start of b/

@@@ → line no & pos^n of change.

---

1) git diff - staged
last commit vs staged diff

---

2) git diff <b1> <b2>
Compare 2 Branches

---

3) git diff <commit 1>
<commit 2>
diff^n btw 2 commits provide commit hash

Git stash. Save file changes in a temp location useful for saving work before switching branches.

Use case : In conflicting changes not allowed to switch branch as before committing them (stash is alternative)

git stash : curr changes saved in temp location

---

git stash save "name"
named stash

---

git stash list list the stash

---

git stash apply applied most recent stash

---

git stash apply st1 applied st named stash

---

git stash pop applied & Remove

---

git stash clear Remove all

git tag → Name | tage the commit

git tag <xyz> → xyz name to curr commit

git tag → list all tags

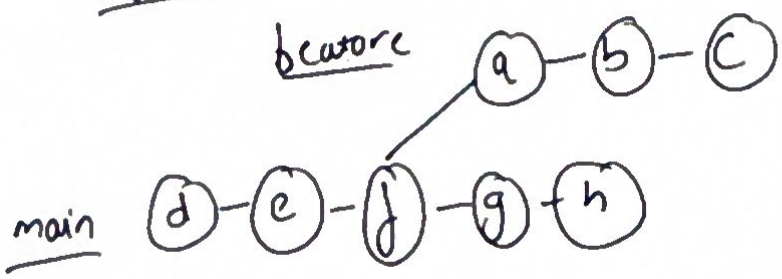
git tag <tag name> → speific commit named
<commit hash>

git tag -d <tagname> → delete a tag

MANaging History

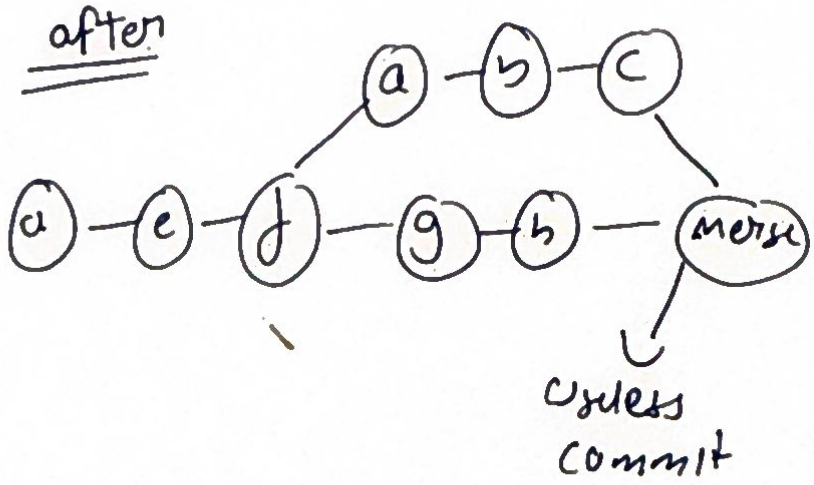
Git Merge

Before



git checkout main
git merge feature

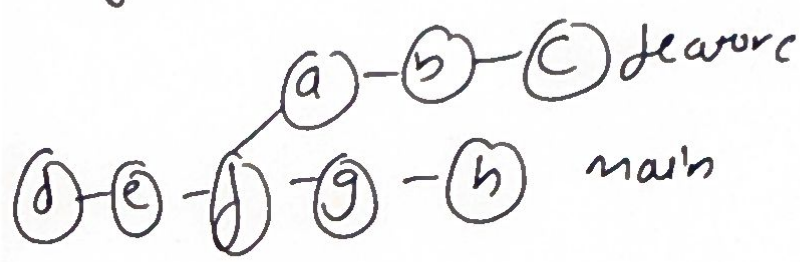
after



2) dirty history

Git Rebase

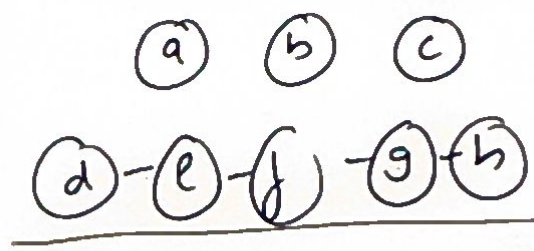
Before



git checkout feature
git rebase main

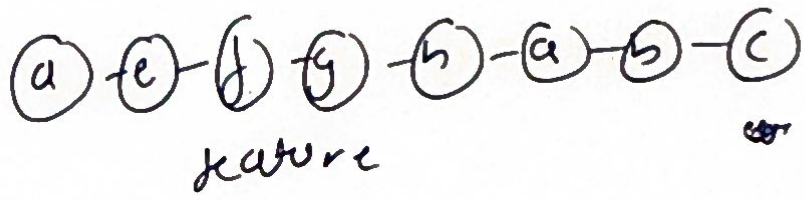
---

(1) Temporary removes all feature commits & saves them as file



---

(2) Then replay them after last main commit-



GITHUB

(1) Make a .gitignore
→ ni .gitignore
or make it manually in folder

(2) git remote add origin "repo link"

(3) git branch -M main

(4) git push -u origin main

(1) git remote -v
show remote url of or Local Repo

---

upstream set
(1) git push -u origin main

maps: main --> origin/main

(2) git push or git pull
↳ from next time

Fetch → Download but dont update local files

(Ex) Github A → B → C
Local A → B

Run: git fetch origin

NOW in git A → B → C
Local A → B

(No code change)

Pull Download Plus update
git pull = git fetch + merge

after pull
Local = Remote